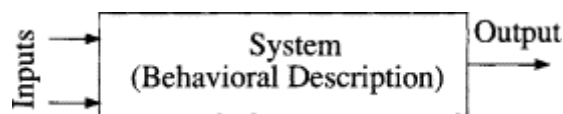


Descripción de Alto Nivel

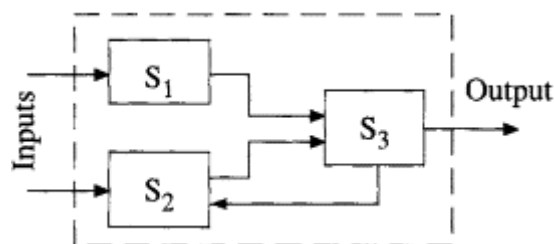
Principios de Diseño

El proceso de diseño generalmente se fundamenta en la metodología de ingeniería para la resolución de problemas. Generalmente, el top-down sigue la siguiente secuencia:

1. **Entender el problema:** se requiere un levantamiento de requisitos del sistema, que definen las entradas, salidas, control y el comportamiento del sistema. Asimismo, define aspectos físicos y eléctricos del sistema en cuestión, como la tecnología, energía de operación, entre otros.
2. **Levantamiento de las interfaces:** consiste en enumerar las entradas, las salidas y el control del sistema, como si se tratase de una caja negra. Podemos llamar a este diseño como el diagrama de nivel 1, donde todo se ve como si fuera una caja negra.

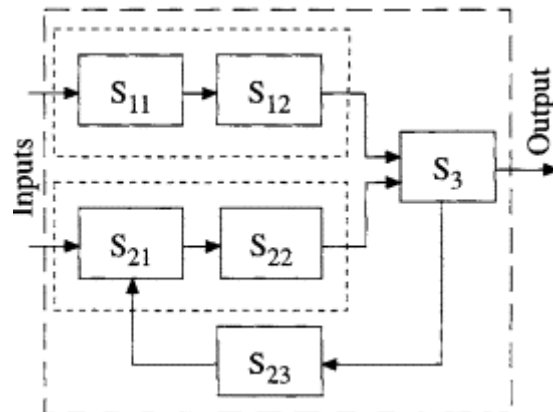


3. **Definición del funcionamiento interno:** consiste en abrir la caja negra y comenzar a incorporar una arquitectura en el sistema, que se puede ilustrar como un diagrama de bloques funcionales. En este punto, componemos la arquitectura como una ruta de datos (*datapath*), que realiza transformaciones de los datos de una forma casi funcional. Tomaremos de ejemplo que S3 corresponde a un sumador.



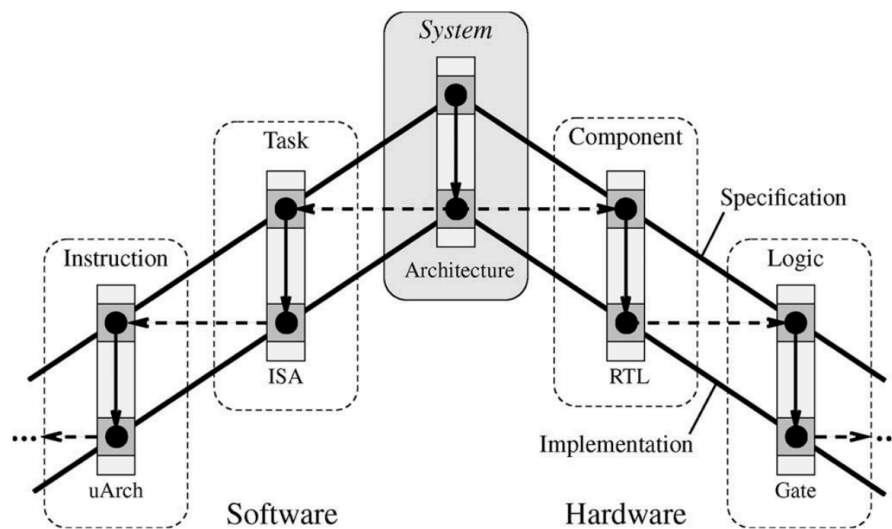
4. **Definición de la implementación:** cuando la funcionalidad está descrita mediante una arquitectura, el siguiente paso es la microarquitectura, que define cómo se implementará cada bloque para brindar la funcionalidad requerida. Un ejemplo de ello es que existen distintas implementaciones para un sumador, como el sumador

de *ripple carry* o de *carry look-ahead*. También, es permitido dividir los bloques funcionales en bloques más pequeños.



Figuras adaptadas de: Lars Wanhammar. 1999. DSP Integrated Circuits

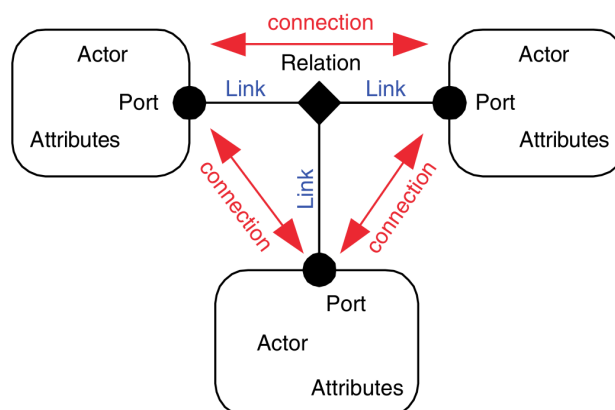
Idea Principal



Flujo de ESL. En este caso, la especificación del modelo de sistema (centro) permite el desarrollo en ambas direcciones. Obtenido de: Gerstlauer et al. 2009. Electronic System-Level Synthesis Methodologies.

La descripción de alto nivel consiste en la especificación de un sistema o arquitectura mediante la abstracción de tareas (del lado del software) y de componentes (del lado del hardware).

Para lograr un alto nivel de abstracción, un sistema está compuesto de componentes, que van a ser actores, con atributos y puertos:



MoC Framework. Obtenido de: Bailey et al. 2010. ESL Models and Their Applications.

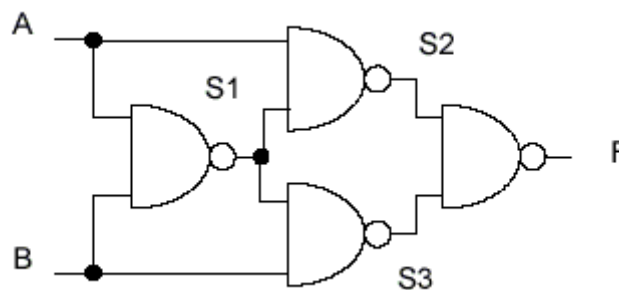
Cada componente tendrá una tarea (o task) que se ejecutará emulando el software implementado a nivel funcional, con un nivel de abstracción más alto que el de la implementación final. Esa tarea permite que el componente se comunique con otros componentes mediante puertos y enlaces.

SystemC

SystemC es un framework basado en C++ que permite la representación funcional de hardware y software. Es un framework que unifica ambos componentes (tareas y componentes) para representar una arquitectura a nivel transaccional.

Representación de lógica combinacional

Basado en: [Duolos](#)



```
#include <systemc.h>

SC_MODULE(nand2)          // declare nand2 sc_module
{
    sc_in<bool> A, B;      // input signal ports
    sc_out<bool> F;        // output signal ports

    void do_nand2()        // a C++ function
    {
        F.write( !(A.read() && B.read()) );
    }

    SC_CTOR(nand2)         // constructor for nand2
    {
        SC_METHOD(do_nand2); // register do_nand2 with kernel
        sensitive << A << B; // sensitivity list
    }
};
```

Primeras impresiones:

- Los componentes se describen como **módulos**, que son nombrados, tal como ocurre en lenguajes HDL.

- Declaramos atributos como **entradas y salidas utilizando sc_in y sc_out**, cuyos tipos de puerto se especifican mediante el template.
- Especificamos un **constructor** que permite **registrar los métodos** que se ejecutarán (ejecución de la transacción) y la **lista de sensibilidad**, que activará el método registrado cada vez que se detecte un cambio en las entradas.
- Los métodos nos permiten representar transacciones.

En este caso, vemos una compuerta NAND como un circuito combinacional.

Para importar módulos dentro de otros módulos.

```
#include <systemc.h>
#include "nand.h"

SC_MODULE(exor2)
{
    sc_in<bool> A;
    sc_in<bool> B;
    sc_out<bool> F;

    nand2 n1, n2, n3, n4;

    sc_signal<bool> S1;
    sc_signal<bool> S2;
    sc_signal<bool> S3;

    SC_CTOR(exor2) : n1("N1"), n2("N2"), n3("N3"), n4("N4")
    {
        n1.A(A);
        n1.B(B);
        n1.F(S1);

        n2.A(A);
        n2.B(S1);
        n2.F(S2);

        n3.A(S1);
        n3.B(B);
        n3.F(S3);

        n4.A(S2);
        n4.B(S3);
        n4.F(F);
    }
};
```

En este caso, construimos cada módulo y le asignamos un nombre a cada uno para mejorar la trazabilidad. Conectamos los módulos a sus señales en el constructor.

Representación de lógica secuencial

La lógica secuencial siempre conlleva el uso de un reloj para su comportamiento en el tiempo. Un ejemplo clásico es el contador binario, que puede representarse de la siguiente manera:

```
#include <systemc.h>

SC_MODULE(counter4)
{
    sc_in<bool> Clk;
    sc_in<bool> Reset;
    sc_in<bool> Enable;
    sc_out<sc_uint<4>> Q;

    sc_uint<4> count;

    void count_proc()
    {
        if (Reset.read())
        {
            count = 0;
        }
        else if (Enable.read())
        {
            count = count + 1;
        }

        Q.write(count);
    }

    SC_CTOR(counter4) : count(0)
    {
        SC_METHOD(count_proc);
        sensitive << Clk.pos();

        // Avoid extra-startup at time 0
        dont_initialize();
    }
};
```

Para este caso particular, la única entrada sensitiva es el reloj y únicamente en los flancos (.pos() = positivo, .neg() = negativo). Si se agrega cualquier otra entrada, el circuito secuencial tendrá un comportamiento asincrónico en dicha entrada.

Adicionalmente, es relevante destacar los tipos de precisión arbitraria de SystemC:

```
sc_out<sc_uint<4>> Q;

sc_uint<4> count;
```

Se pueden especificar enteros de cuatro bits o de cualquier otra precisión.

Construcción de modelos de verificación (no estándar)

Para propósitos de verificación, es posible preconstruir el verificador de la implementación de alto nivel. Para ello, se considera:

1. Un estimulador o driver que genera los estímulos de módulo a verificar (DUT).
2. Un monitor o scoreboard que monitoriza las respuestas del módulo a verificar (DUT).

```
#include <systemc.h>

SC_MODULE(stim)
{
    sc_out<bool> A;
    sc_out<bool> B;
    sc_in<bool> Clk;

    void StimGen()
    {
        A.write(false);
        B.write(false);
        wait();
        A.write(false);
        B.write(true);
        wait();
        A.write(true);
        B.write(false);
        wait();
        A.write(true);
        B.write(true);
        wait();
        sc_stop();
    }
    SC_CTOR(stim)
    {
        SC_THREAD(StimGen);
        sensitive << Clk.pos();
    }
};
```

El estimulador debe tener una noción temporal para realizar transiciones y estimular distintas combinaciones de entradas. Eso lo agregamos con el reloj. Note que esto se realiza mediante un SC_THREAD, que se detiene hasta detectar otra invocación del reloj.

El monitor tiene una similitud con el estimulador, el cual, también, requiere una noción de tiempo si se quiere monitorizar mediante muestreo. Si se nota, también tiene un SC_THREAD que queda en un ciclo con wait para bloquear la ejecución hasta el próximo estímulo.


```

#include <iomanip>
#include <systemc.h>

SC_MODULE(mon)
{
    sc_in<bool> A;
    sc_in<bool> B;
    sc_in<bool> F;
    sc_in<bool> Clk;

    void monitor()
    {
        std::cout << std::setw(10) << "Time";
        std::cout << std::setw(2) << "A";
        std::cout << std::setw(2) << "B";
        std::cout << std::setw(2) << "F" << std::endl;
        while (true)
        {
            std::cout << std::setw(10) << sc_time_stamp();
            std::cout << std::setw(2) << A.read();
            std::cout << std::setw(2) << B.read();
            std::cout << std::setw(2) << F.read() << std::endl;
            wait(); // wait for 1 clock cycle
        }
    }

    SC_CTOR(mon)
    {
        SC_THREAD(monitor);
        sensitive << Clk.pos();
    }
};

```

TLM 2.0: Transaction-Level Modeling version 2.0

El modelado de nivel transaccional es un estándar (establecido por the Open SystemC Initiative) para construir System-on-Chips (SoCs) interoperables, de alta velocidad y con prototipos virtuales en C++.

En vez de modelar señales individuales (clk, data, enable, etc.), TLM modela transacciones completas, por ejemplo:

- “leer 4 bytes desde la dirección 0x1000”
- “escribir un paquete en memoria”

La idea es simular más rápido, reducir la complejidad y permitir una exploración arquitectónica temprana de forma más rápida y eficaz.

En TLM, vamos a tener las siguientes partes:

- Iniciador (Initiator): es quien inicia las transacciones.
- Objetivo (Target): quien responde a las transacciones.
- Socket: conexión entre módulos.
- Payload: objeto que contiene la transacción.
- Interfaz de transporte: método usado para enviar las transacciones.

Iniciador

El iniciador es quien inicia las transacciones. Puede verse como el maestro en la comunicación, quien gobierna cuándo se inician las transacciones.

Generalmente, el iniciador es un módulo de SystemC con un thread que le permite iniciar las transacciones y un método de envío de transacciones. Para el envío de transacciones, se necesita declarar un socket, preparar el payload y enviarlo.

```

#include <systemc>
#include <tlm>
#include <tlm_utils/simple_initiator_socket.h>

struct CPU : sc_core::sc_module
{
    tlm_utils::simple_initiator_socket<CPU> socket;

    SC_CTOR(CPU) : socket("socket")
    {
        SC_THREAD(run);
    }

    void run()
    {
        std::uint32_t write_value = 0x1234ABCD;
        send_transaction(tlm::TLM_WRITE_COMMAND, 0x10, write_value);

        std::uint32_t read_value = 0;
        send_transaction(tlm::TLM_READ_COMMAND, 0x10, read_value);

        std::cout << sc_core::sc_time_stamp() << " CPU read value = 0x"
        | | | | | << std::hex << std::setw(8) << std::setfill('0') << read_value
        | | | | | << std::dec << std::setfill(' ') << std::endl;

        sc_core::sc_stop();
    }

    void send_transaction(tlm::tlm_command cmd, std::uint64_t addr, std::uint32_t& value)
    {
        tlm::tlm_generic_payload trans;
        sc_core::sc_time delay = sc_core::SC_ZERO_TIME;

        trans.set_command(cmd);
        trans.set_address(addr);
        trans.set_data_ptr(reinterpret_cast<unsigned char*>(&value));
        trans.set_data_length(sizeof(value));
        trans.set_streaming_width(sizeof(value));
        trans.set_byte_enable_ptr(nullptr);
        trans.set_dmi_allowed(false);
        trans.set_response_status(tlm::TLM_INCOMPLETE_RESPONSE);

        socket->b_transport(trans, delay);
        wait(delay);

        if (trans.is_response_error())
        {
            SC_REPORT_ERROR("TLM-2", trans.get_response_string().c_str());
        }
    }
};

```

Nótese cómo se describe el módulo. Esta vez no se usa el wrapper de SC_MODULE, pero se importan nuevos headers para utilizar las herramientas de TLM.

Objetivo

Se comporta como el slave en la comunicación, ya que solo recibe transacciones y las ejecuta. Un ejemplo habitual es una memoria, que recibe transacciones de lectura y escritura.

```
#include <systemc>
#include <tlm>
#include <tlm_utils/simple_target_socket.h>

struct Memory : sc_core::sc_module
{
    tlm_utils::simple_target_socket<Memory> socket;

    static constexpr unsigned int kSize = 256;
    std::array<unsigned char, kSize> data{};

    SC_CTOR(Memory) : socket("socket")
    {
        socket.register_b_transport(this, &Memory::b_transport);
    }

    void b_transport(tlm::tlm_generic_payload& trans, sc_core::sc_time& delay)
    {
        const auto cmd = trans.get_command();
        const auto addr = trans.get_address();
        auto* ptr = trans.get_data_ptr();
        const auto len = trans.get_data_length();

        if ((addr + len) > kSize)
        {
            trans.set_response_status(tlm::TLM_ADDRESS_ERROR_RESPONSE);
            return;
        }

        if (cmd == tlm::TLM_WRITE_COMMAND)
        {
            std::memcpy(&data[addr], ptr, len);
        }
        else if (cmd == tlm::TLM_READ_COMMAND)
        {
            std::memcpy(ptr, &data[addr], len);
        }

        delay += sc_core::sc_time(10, sc_core::SC_NS);
        trans.set_response_status(tlm::TLM_OK_RESPONSE);
    }
};
```

Note que se pueden incluir aspectos de tiempo para modelar imperfecciones de los módulos y su velocidad.

```
#include <systemc>
#include "cpu.h"
#include "memory.h"

int sc_main(int argc, char* argv[])
{
    (void)argc;
    (void)argv;

    CPU cpu("cpu");
    Memory memory("memory");

    cpu.socket.bind(memory.socket);

    sc_core::sc_start();
    return 0;
}
```

Para unir los módulos, se tiene que hacer bind para efectuar la conexión entre los módulos de la arquitectura/sistema.

Tareas:

1. Replicar los snippets de esta lección en su ordenador.
2. Comenzar investigación de Evaluación Corta 1 en los grupos correspondientes (disponible en TEC Digital).

Esta lección se abordará brevemente en la siguiente lección.